Supplement: signals, interfaces and worked-example data

Jonathan Matkowsky

29 October 2026 · Supplement to preprint version 1.0 · Not peer reviewed

This supplement accompanies the complete manuscript. S1 and S2 explain the sources and selection. S3 to S5 specify the signals, records and interfaces. S6 supplies the hypothetical investigations and their arithmetic; S7 and S8 give the proposed evaluations. S9 extends the comparison with prior work. The 19 CSV files use UTF-8 encoding and stable mechanism, source and signal identifiers. Citation labels match the package's BibLaTeX references.

S1. Sources and comparisons

Source notes identify the version, supporting passages, publication status and supported claim for each of the 122 cited works. The requirements matrix compares 50 sources against C1 to C3. It codes two parts of C1 and C2 and three of C3: an outcome linked to its decision, and late, unobservable and false reports kept apart (C1); query coverage and learned time (C2); typed separation of observations, inferences and memory, a human principal before reuse, and links from records an agent output informed back to that output (C3). E means explicit support, P partial or adjacent support, and a dot means the cited material does not establish the requirement. The `payment_setting` column records yes for a payment or payment-fraud setting, P for a setting partly in payments or adjacent to payment fraud, and no otherwise. It marks two cyber threat intelligence sources, STIX 2.1 and Hettema's belief-revision paper, as cyber; other security and threat-intelligence sources are coded no. The source notes provide passage locators, linked to the matrix by citation key. Their inspection dates are in UTC.

Stage coverage supplies the source-level codes behind manuscript Table 1. Each family receives the strongest code among its sources, so a family row may combine capabilities from several systems. Source notes record bibliographic and access issues separately from claims about a method.

Study populations and denominators constrain the comparisons. Sharma's balanced synthetic sample contains 60 cases drawn from transactions its classifier scored between 0.2 and 0.8: thresholding the classifier at 0.5 decides 43 correctly and the agent 39. Six of the agent's eight overrides replace a correct decision with an error. Fatima's verification rates concern genuine participants under the study's completion and exclusion rules; they do not measure fraud detection. Doerfler's 47.8 percent same-session success rate concerns challenged sign-ins. Its 97.3 percent eventual-access rate counts users, not sign-ins: it is the share of users with access within one week. Doerfler's Section 5.3 and Table 7 caption take users who failed a challenge as the denominator; Doerfler's introduction takes users who saw a challenge. Neither Fatima's nor Doerfler's aggregate rate establishes whether an individual event was decided correctly.

FTC complaints describe allegations, and the stipulated orders do not establish those allegations on their merits. Vendor publications support attributed descriptions of interfaces or observations, not independent evidence of industry-wide effectiveness.

S2. Product scope and mechanism selection

Product scope contains 88 public product and feature entries, read on 27 September 2026: 53 catalog items, twenty items from the Q3 2026 public roadmap (seventeen appear in a cached copy of that roadmap; the other three are confirmed as shipped in Stripe's changelog), ten separately documented capabilities and five careers-page footer items. These entries define the scope for

candidate selection, not a complete or current product catalog. Each row lists candidate mechanism IDs, not validated coverage or retained signal mappings; a dot means none is listed. The allocation table gives each candidate's final disposition, and S3 supplies the retained mappings. Six retained mechanisms are not listed against any product entry: ID-14 (account farming and aged-account preparation for later abuse), PAY-04 (card testing through attacker-controlled merchant or connected accounts), MP-02 (payment-method cashing through a self-controlled merchant or connected account), MP-04 (merchant misrepresentation and transaction laundering behind a front), MP-07 (merchant-side dispute evasion) and BOT-04 (CAPTCHA and challenge defeat). S5 identifies the exact API schemas used for interface comparisons.

A retained mechanism targets a documented capability, serves a plausible adversary goal and has supporting public evidence. Sources include research, incident reports, government and intergovernmental guidance, and platform documentation. Job postings helped prioritize topics but do not establish incidents. Every mapped mechanism has support beyond postings in the mechanism-to-source links.

Mechanism allocation records all 99 candidates and their final dispositions. The signal map retains 61 mechanisms: 52 of the 53 dedicated candidates, all eight targeted candidates and the one adjacent-logged candidate, ID-18, insider access abuse by platform or partner staff. Of the other 38, thirteen were judged covered by existing records, four were screened without identified supporting evidence and logged as low-priority gaps, twenty low-priority candidates were not reviewed, and one reviewed candidate, PAY-12, has no signal mapping. It concerns triangulation fraud: filling real customers' orders through another seller using stolen payment credentials. Its review record notes framework evidence only. Allocation keys decode the source abbreviations. The `Class` code, also the prefix before each key's colon, gives the source type: a for law-enforcement announcements and vendor or incident reports, including EchoLeak (a:ECHO), which is cited by its arXiv identifier; b for research papers; b* for other arXiv preprints; c for government and intergovernmental publications and the MITRE fraud framework; d for the illustrative platform's documentation, roadmap, blog and public repositories; and e for the illustrative platform's job postings (Stripe careers). All four b keys name a peer-reviewed venue. In the `Verification` column, V marks a key recorded as verified, and S marks a key that rests on a summarizing retrieval whose exact passage was not retained, as the `d:S` row states. The allocation file repeats S as a suffix on its evidence keys, as in `a:SSN (S)`. Priority labels explain selection emphasis; they do not measure evidence quality.

Reading the selection records The allocation file's `disposition` field states the final selection decision. Signal coverage is recorded separately in S3; a reviewed candidate need not have a signal mapping.

Disposition	Meaning
dedicated	Reviewed as a distinct mechanism
targeted	Received a focused review of its implications for the design
covered	Judged covered by existing records rather than retained as a separate mapped mechanism
screened gap	Screened without identified supporting evidence and logged as a low-priority gap
unreviewed	Not reviewed; all twenty such candidates have low priority
adjacent logged	Recorded as adjacent to the core mechanisms; the one such candidate is retained in the signal map

`Priority (basis)` and `Impact (basis)` retain the selection judgments and their stated reasons. In a priority basis, `r:` introduces a topic named in a job posting, with the posting's key in parentheses where given, and `p:` introduces platform-documentation keys. A posting key such as `(EM)` is the allocation key `e:EM` without its class prefix; platform-documentation keys keep their `d:` prefix. Impact bases use no prefixes. `Scope` distinguishes core and adjacent mechanisms. `Parent` records a stated relationship to another mechanism; a dot means none is listed. `Evidence keys` resolve to source descriptions in the allocation-keys file. They are identifiers, not evidence grades. `Maturity` is the estimate, before review, of how developed the published research was; `after_review_maturity` pairs it with the later classification where both were supplied. `Evidence` is the confidence label used to allocate effort (verified, provisional or unresolved), not a certification of each source's claims. `Depth` is the initial allocation: Tier 1 for the deeper review, Tier 2 otherwise, or a logged gap when evidence was unresolved. `recorded_status` states each review's outcome. The product-scope file links documented capabilities to candidates; it does not supply a product-availability assessment.

Searches covered nine abuse paths, coded U1 to U9: API-key takeover (U1), identity provenance at onboarding (U2), document integrity, including identity and business documents (U3), account compromise (U4), payout redirection (U5), card testing (U6), abusive merchants (U7), mule and pass-through funds (U8), and attacks on agents or operator assistants (U9). Only the entity-relationship notes cite these codes. Searches followed relevant publications, public guidance and references. This focused selection does not establish exhaustive coverage of abuse or related systems.

S2.1 Claim types

Research can propose an untested design, while documentation can directly establish an interface. Neither publication status nor the number of sources validates the integrated specification. No claim in either document rests on author experience.

Table S2.1. How to read the manuscript's claims. Claim type describes what a statement establishes. Source type and publication status describe where its support comes from.

Claim type	What it means
Fact	A description supported by evidence, including a public specification or an artifact this study produced; source and version are given where they matter
Peer-reviewed prior research	A finding reported in a peer-reviewed publication whose publication status was confirmed, applicable only to the setting and conditions it studied
Inference	A conclusion motivated by stated premises; the sources do not establish the conclusion directly
Design requirement	A property chosen for the proposed platform on the stated evidence and reasoning; it is not claimed as necessary for every platform or as observed in a deployment
Hypothesis	A proposition to test for the proposed platform, including every detection candidate in the manuscript
Proposed implementation	One possible way to satisfy a requirement
Empirically validated result	A measurement, valid for the proposition and setting actually tested; results for individual components do not validate this integrated design

S2.2 External coverage check

An external check compares the candidates with MITRE's Fight Fraud Framework (F3), version 1.1 [1]. The F3 crosswalk maps each of its 123 techniques and sub-techniques to the candidate

mechanisms that cover it, with a reason for each scope decision. Of the 92 within the product scope, a retained mechanism covers 34 fully and 40 in part. Of the other 18, fourteen are covered only by candidates outside the signal map. Five of the fourteen rest only on candidates judged covered by existing records, among them device fingerprint spoofing; six rest only on unreviewed candidates, among them 3-D Secure bypass; and three rest on candidates of both kinds. None rests on a screened gap. Four are covered by no candidate: DLL injection in the browser, abuse of accessibility features, theft from password managers and SEO poisoning. Scope and coverage are one reviewer's judgments, recorded for each row.

S3. Signals, mappings and lineage

The signal inventory defines all 71 signals, with their families, classes, feature inputs, mechanism and source IDs, public counterparts, entities, uses and limits. Its `internal_representation_status` value, DR for every signal, marks a design requirement rather than observed platform instrumentation. Mechanism-to-signal mappings contain 163 links: 154 design mappings motivated by evidence and nine marked as inferential. These links explain why a record is proposed; they do not establish a detector's effectiveness or make its fields uniquely necessary.

The outcome-channel requirements use SIG-70 and SIG-71 to record each payment-level or account-level control decision's effect on each outcome channel, SIG-27 to separate authentication results from reported liability status, and SIG-58 for dispute reasons and payout returns. SIG-70 also keeps manual-review dispositions. Each record retains its source, version and uncertainty.

The inventory contains 26 source observations (SO), two normalized observations (NO), 24 derived features (DF), two model or risk outputs (MO) and ten control outcomes (CO). Seven signals provide external knowledge (XK). The inventory therefore covers five of the seven record classes in manuscript Section 5.1, plus external knowledge. Analytical inferences and finished intelligence are later outputs, not input signals.

SIG-44, SIG-45 and SIG-46 apply across mechanisms. They record technique-catalog mappings, minimum observation provenance and the provenance of test records, respectively. The other 68 signals map to specific mechanisms. All 61 mapped mechanisms have at least one mapping motivated by evidence; S2 explains PAY-12's exclusion.

Each input ID for the 24 derived features names another signal in the inventory. No feature depends, directly or indirectly, on itself. For example, SIG-24 uses SIG-27 to enumerate its eligible transaction cohort, including completed payments with no report. The separate chargeback-or-alert-resolution measure also uses SIG-25. Keeping these inputs explicit makes the denominator and the distinction between measures inspectable.

The source directory identifies 61 design sources. Of these, 27 are not among the 122 cited works and have no S1 source note. Each source was read in October 2026: 60 in full and one, an FTC complaint, in part. For one of the 60, a vendor blog post (L112), the cited passages were read verbatim and the other paragraphs only as a close paraphrase. The directory records each source's type, the depth and version of the read, with qualifiers such as the version read or pages not read, the access date and the scope. Where only a preprint or an author copy was read, the row says so. Mechanism/source joins supply 126 links, each with its source identifier, recorded depth and scope limit. These records preserve the premises behind the mappings, including limits on transferring evidence from another setting.

The public-counterpart classification asks whether a pinned schema contains an object, field or event that supplies part of the required observation. Of the 71 signals, 32 have partial counterparts, the

question does not apply to 32, and none was identified for seven. S5 gives the exact locators and explains the limits of partial counterparts; the inventory records each signal's classification.

Identifier links establish limited relationships. A storefront-to-merchant identifier link does not show that a payment through the merchant account settled. A verified identifier from the same namespace and relevant interval can support a stronger, or *hard*, link. A descriptor or shared IP address may support only a weaker, or *soft*, link. Neither alone establishes common control or malicious intent.

The inventory preserves the manuscript's interpretation rules: onboarding-link histories are a design analogy (Section 5.2), and error estimates retain their cohorts and denominators (Section 7.2). Each outcome channel retains its decision context (Section 7.1), and each broader outcome measure requires its own evaluation (Sections 8.2 and 9.1).

S4. Responsibilities, entities and interfaces

The functional responsibilities file defines 24 responsibilities without requiring separate processes or databases. Seventeen were derived with payment-specific mechanisms or sources in view. One (23) adapts agent-security research, and six (2, 3, 4, 13, 21 and 22) come from cyber threat intelligence work. The file's `class` column marks these three groups DI, AGV and INH.

Responsibilities 16 and 17 cover engineering validation and adaptive testing of detections, controls and models. Responsibility 21 independently reviews all agent outputs, including intelligence drafts, hunting and detection candidates, and recommendations; responsibility 22 provides human release. Permission to investigate grants no authority over accounts, payments or controls. An authorized human may decide urgent containment before quality review (manuscript Section 6.4), without releasing the draft into organizational memory.

The entity inventory and entity relationships define the entities and the kinds of links between them. Each link records its source, identifier namespaces and the basis for associating each identifier with its entity. It retains its effective interval, learned time, version, confidence, confounders (other explanations for the link), and permitted purpose and scope. A shared attribute alone does not establish that one person controls both entities. When a decision makes an outcome unobservable, the record describes the channel's status; it does not invent an outcome report. S4.1 gives the complete storage requirements.

The query interface must show what each query actually covered and what its result means:

Part of the interface	Information or behavior required
Request	Task and question; authorized requester; tenant, purpose and data scope; entity namespaces; event interval and as-of cutoff (only data the system had learned by then); schema and query versions; row, time and cost limits; expected observation and required coverage
Response	Query/run ID; scope actually evaluated; source and schema versions; event and ingestion watermarks (how far event time and data loading had progressed); known gaps and retention horizon for each source; sampling and access filtering; whether the result is complete, partial, truncated, denied or unavailable, and any scope-limited flag; meaning of any count; returned rows or approved aggregate, with the identifiers of the returned records kept for the run; lineage and coverage limits

Part of the interface	Information or behavior required
Interpretation	A positive observation does not establish how many cases exist in the population. An empty result counts against a hypothesis only when the abuse should have produced the queried observation and the records cover where it would appear. A denied or unavailable result does not establish adversary behavior.
Failure	Enforce limits outside the agent's language model. Stop or narrow the query after it exceeds a row, time or cost limit, or after an authorization error. Record incomplete tests and any abstention (a decision not to conclude). Do not treat missing or not-assessed data as low risk.

Some permitted operations also need approval bound to the exact request payload, target, task, tenant, tool and tool-manifest version. The manifest is the declared list of the agent's tools. Each approval has an expiry and a rule governing reuse. At execution, the enforcement gateway (manuscript Section 6.5) checks that the operation matches the approval and remains permitted. The restricted agent-and-approval ledger records both approval and execution.

Control and outcome records distinguish the decision, its inputs and version, what happened next, the observable channels, actual reports and any review of correctness. Population error estimates retain their own cohort, denominator, outcome window, uncertainty and scope; they do not establish correctness for individual events. Late reports, unobservable outcomes and false reports have separate fields because they can coexist. Report presence, channel state and query access also remain separate. Each temporary delay retains its decision and effective interval. A derived fact's learned time is when it became queryable, kept separately from its inputs' latest availability.

A hypothetical timeline illustrates the as-of cutoff. Freeze a prediction on day 0 using only information then known. A day-30 report may evaluate it within a stated outcome window, but a model trained on day 20 cannot use that report. A later training run may use it if the run's cutoff and label rules permit. Likewise, a relationship learned on day 10 cannot enter day-0 features merely because its effective date is earlier.

The draft schema supplies the fields that the rule checker, quality reviewers and the releaser use to assess an intelligence draft. S7 explains the evaluation conditions; S7.1 gives hypotheses and criteria for E1 to E9. S8 gives the practitioner review protocol.

S4.1 Complete storage requirements

Manuscript Table 3 summarizes this complete storage contract; manuscript Section 5.2 gives the motivating evidence. Each block groups requirements by the information it preserves. The stores distinguish record meanings and need not be separate databases.

Observation log

- Preserve records and revisions without overwriting earlier records. Keep source, event and delivery identifiers, schema version, entities, source lineage, and actor and request identifiers where available. A record delivered more than once takes effect only once.
- Record each observation's environment (production, test, synthetic, replay or emulation), marked when the record is created, and its tenant scope: the business or other boundary within which it may be accessed and used. Keep each record's source trust mark. Mark content that businesses, customers or outside sources supplied as untrusted. Both marks survive normalization and storage.

- For content fetched from a source the subject controls, such as a storefront or business website, record the vantage point of each fetch: network origin, region and client. A subject can serve different content to different visitors, including reviewers.
- Mark every label, assessment, control outcome, draft, proposed lesson and released item that a test, replay or emulation creates, or that derives from such a record or from synthetic data, with the same environment value when it is created. Keep these records out of production labels, intelligence and lessons. Record any authorization for their evaluation use, with who gave it, the evaluation and its scope.
- Keep event or effective time (when something happened or took effect) with its source and uncertainty bound. Record the source system's recorded time and the platform's ingestion time separately.
- For each store, retain when each record and revision became visible to queries.
- A derived record keeps its inputs' latest learned time separately from the time it first became queryable. Its own learned time is the latter and cannot precede any input's learned time.

Entity-relation graph

- Label each identifier link by kind and as hard (stronger) or soft (weaker). Retain its source, identifier namespaces, effective interval, learned time, version, confidence and possible benign explanations (confounders).
- Record why each identifier is believed to belong to its entity.
- Keep versions of linked-entity clusters and their membership rules. Record end customers separately for each business. Record each link's permitted purpose and scope. A link across businesses also records its authorization for that identifier type (manuscript Sections 5.2 and 5.5).

Feature store

- Keep inputs traceable and values recomputable, with the feature definition version, the entities and the computation window. Decision-time features use only information available at that decision.
- A feature computed over a cluster or other threat-knowledge item records that item's version, writer and status. If an agent proposed the item, or the item links to an agent output or released organizational-memory item that informed it, the feature links to that output or item and counts as dependent on it.
- Join later outcomes as evaluation labels under an explicit outcome window and maturity period. The outcome window is the period after the decision in which an outcome must occur to count.

Label store

- **Report history:** retain report type, source, attributed transaction or entity, arrival time and lag. Preserve each value and its learned time, such as a dispute being filed, answered, then won or lost, and record whether each won or lost status followed adjudication.
- **Truth assessment:** link any assessment that the report is mistaken or deliberately false. Keep that assessment in the assessment store with its model or reviewer identity and version, how the report was selected for review and, for a sampled review, its selection probability.
- **Dependencies:** link decisions that affect whether reports can arrive through each outcome channel, and any agent output or released organizational-memory item that informed the label. An outcome channel is a route, such as a chargeback, through which an outcome can be reported. Record the writer's case or session and the agent outputs and released items open in it when the record was written.

- **Channel status:** for each affected transaction, payout or account application, keep report presence, channel state and query access separate at each read time. These statuses may be computed when read from the stored decision, report and channel records. When a draft cites a status, store that status with an identifier and its read time.
- **State history:** a channel is open, temporarily delayed or not yet opened, permanently closed by a recorded decision, reopened or extended by a recorded rule event, or unknown. Keep every transition's effective interval and the decision ID, or the rule event's source and version, that caused it.
- **No report:** an open channel remains pending before its reporting window ends. It matures with no report only if it stayed open throughout the window and the records cover that window. If a recorded decision or rule event changed the channel's meaning, the matured absence carries that decision's ID, or the rule event's source and version, and is read under the changed meaning, not as a negative for the original outcome. A channel with no defined window never supports a negative. A matured absence is provisional: it records the rule version under which it matured and returns to pending if a later rule event reopens the window.
- **Unavailable records:** mark inaccessible records unavailable without overwriting known channel state or earlier reports.
- **Clocks and maturity:** for a temporary delay, keep the expected and actual clock-start events when known. Store the reporting window and clock-start event with the channel, including their source and version. Each channel record also names its rail, the attributes its window depends on, such as whether the debited account is a consumer or business account or the type of claim, and whether the channel adjudicates report truth. Store the maturity used by an evaluation with that evaluation.

Assessment store

- Retain the model or reviewer identity and version. Keep unknown, not-assessed and failed-open states distinct from low risk. Failed-open means the assessment failed and activity continued.
- Record when each assessment was made and frozen (its freeze time) and the as-of time of its inputs. For each model version, record its training and model-selection cutoff and the label rules it used. A label that became available after that cutoff cannot enter that version.
- Never store an assessment as an observation of the conduct it assesses.
- Link each assessment, whether a model, a detection or a reviewer produced it, and each detection or model version, to any agent output or released organizational-memory item that informed it. Record the writer's case or session and the agent outputs and released items open in it when the record was written.
- For each detection or model version, link the validation records of responsibilities 16 and 17, with their version, date and result.

Control-outcome store

- **Decision:** retain the decision, who had authority to make it, the rule or model version, and the reason.
- **Inputs and policy:** log the time of the decision and the inputs it used. Keep versions from which every other input available at that time can be reconstructed. Record the score, threshold or rule that produced the decision, and whether the policy was deterministic and, otherwise, the probability of each alternative. Identify any input that a decider, automated or human, used but that was not logged.

- **Human decisions:** retain who decided, how the case was assigned and what the decider was shown.
- **Dependencies:** link any agent output and any released organizational-memory item that informed the decision, so the decision can be revisited when that item expires or is corrected or retracted. Record the writer's case or session and the agent outputs and released items open in it when the record was written.
- **Correctness:** record reviewed correctness when available, with how the case was selected for review (for example an appeal, a complaint, a sampling rule or review of every case) and, for a sampled review, its selection probability; otherwise record unknown.
- **Error estimates:** keep separate links to each estimate, with its control version, cohort, outcome window, denominator, uncertainty and scope. The cohort is the fixed set of decisions the estimate covers.
- **Channel effects:** record the decision's effect on each outcome channel as closed, delayed, changed in meaning, none or unknown.
- **Restrictions:** distinguish allowing and blocking from restrictions, such as a hold that may later be released.
- **Urgent containment:** for an action decided before quality review (manuscript Section 6.4), record that route, the impact class (none, degraded service or outage), any notification and restoration path, the review deadline and the later review's outcome, including any reversal.

Structured threat-knowledge store

- For inferred relationships, clusters, technique mappings and conclusions, retain confidence and the input and rule versions that produced them.
- Record the writer as a correlation engine, agent proposal or person, and status as proposed, accepted or retracted. Record the person or accountable process that accepted or retracted the item, and when.
- Retrieve an agent-proposed item only when it is marked as an unreleased agent inference. It never counts as corroboration.
- Link an item that a person or correlation engine wrote to any agent output or released organizational-memory item that informed it. Record the writer's case or session and the agent outputs and released items open in it when the item was written. The item counts as dependent on those outputs and items.

Organizational-memory store

- Keep released items only, with the releaser, version hash, released claims and the confidence stated for each, scope, expiry, reviewer findings and originating draft.
- Retain supersession and retraction links, and lineage from every cited observation to its root record. Record every later status record for a released item, such as a monitoring finding, an early expiry, a retraction or a withdrawal, with who or what wrote it and when.
- For benign or exonerating items, keep a re-test date, a source of counterexamples independent of the writer, and the versions of the cited evidence and of any cluster the item concerns. A revision of that evidence or a change in that cluster's membership expires the item early (manuscript Section 6.5). Record each re-test, with its counterexamples' sources, its result and its date.
- Read each item as context or a hypothesis, never as an observation.
- Proposed lessons remain outside organizational memory until testing, quality review and human release. For each one, record its writer, proposal time, queue state, intended scope and version. Record each test against counterexamples, with the counterexamples' sources

and the result. After release, record monitoring findings and any withdrawal, each with its writer and time, and the reason for a withdrawal (manuscript Section 7.3). The requirement for responsibility 23 in the functions file calls a withdrawal rollback; the two terms name the same record.

External-knowledge store

- Retain source, type, version, when the source observed and published the fact, validity window, license and retrieval time. The validity window is the period in which the fact holds. For a catalog of abuse techniques, the version is the fixed version that mappings cite.
- Keep external knowledge distinct from platform observations.
- Record the identifier attached to each released item shared outside the platform. When external content returns with that identifier, link it to that item. It counts as dependent on the item and on any agent output behind it, not as independent corroboration.

Agent-and-approval ledger

- For each run, record the human assignment or the standing task template and version that authorized it, with its permitted data, entities, period, current period and outputs. The current period bounds claims about current conduct.
- For each tool call, preserve the provenance of inputs that reached the agent's language model.
- For each approval, keep the approver, the target, task, tenant and tool it is bound to, its expiry, its reuse rule and any token issued. Keep hashes of the approved and executed payloads, and the tool-manifest version and hash in force at approval and at execution. The manifest is the declared list of the agent's tools.
- Record memory writes and decisions on outbound traffic.
- Record each agent output, such as an intelligence draft and each of its versions, a hunting or detection candidate or a recommendation, with its identifier, version hash, run and creation time. It is an observation of what the agent did, not evidence of the activity under investigation.
- Retain model identity and version, instruction version and decoding settings.
- Mark content that came from untrusted input as untrusted when it is read back. Distinguish copied untrusted content, agent output exposed to untrusted content and agent output without that exposure; exposure takes precedence.
- Record each rule-checker result and quality-review finding with the draft's version hash, the rule version or reviewer, including the version of the rule that marks uncited returned records, and the draft's resulting status: returned, released or expired. For each quality review, record the materiality tier applied to each claim, the sampling rate, the cited records the reviewer examined and the random sample of unmarked uncited returned records it drew, whose size is fixed in advance for each claim type. Record when each draft version entered the review queue and when it would expire from it. For each review, record its start and end times.
- Record each removal of an agent-output or released-item link from a label, control decision, detection, assessment or threat-knowledge item, with the person who removed it, that person's release authority, the time, the reason and the identifier of the removed output or item.
- Store references to card data, identity documents and other regulated data, not copies (manuscript Section 5.5).

Query coverage record

- For each query run, retain its request: the task, run and question, the authorized requester, the tenant, purpose and data scope, the query version, the row, time and cost limits set, the expected observation and the required coverage.

- For each query run, retain the coverage record the gateway attached: requested and evaluated entities with their tenant and identifier namespaces, requested and evaluated event intervals, as-of cutoff, source and schema versions, sampling, each source's event and ingestion watermarks, known gaps and retention horizon, access filtering, and any row, time or cost limit reached.
- Keep the derived result status (complete, partial, truncated, denied or unavailable), the version of the rules that derived it and any scope-limited flag. The agent cannot write these fields.
- For each query run, keep the identifiers of the records it returned, including those behind an approved aggregate, so that the checker can find returned records that a draft neither cites nor lists against a claim, attach those its fixed rule marks and summarize the rest (manuscript Section 6.4). Keep identifiers, not copies, under each record's retention and deletion rules (manuscript Section 5.5).
- Link each tested hypothesis and each claim of absence to the query runs behind it. Record each hypothesis's status, including inconclusive and unassessable, apart from the query results. Record each incomplete test and each abstention, a decision not to conclude.

Time semantics

- Use timestamps precise enough to join a key's exposure to its use within minutes, with each timestamp's clock source.
- For each payment, keep authorization, capture and processing times separately, and any service or event date that a channel rule uses as its clock start. Keep the authentication result and the reported liability status separately, each with its source, rule version and learned time. Keep any network program status that removes the liability shift as it stood at decision time. Record a later program entry as a rule event, and leave the liability status of payments authenticated before it unknown. An unknown liability status remains unknown.
- Preserve label delay and what information was available as of each time.
- Retain account age, dormancy and how long funds stay before moving on.
- For each record class, keep each record until the longest applicable reporting window, counted from its channel's clock-start event, or the defined maturity period has ended, whichever ends later, plus the investigative lookback period. Follow the retention and deletion rules in manuscript Section 5.5. Record resulting gaps as coverage limits. Record each authorized hold with its scope and retention rule.

Graph semantics

- Link closed accounts to suspected successor accounts, and storefronts to merchants.
- Link each outcome report to the transaction, payout or account application it concerns, with the basis for the link and whether it is verified. Attribute outcomes through that link to the platform account, connected account and end customer, so that a measure counts each affected transaction once. Keep each transaction's acquiring bank and the static part of its statement descriptor, so that a network's monitored unit can be reconstructed.
- Link each wallet to every party it has allowed to spend its funds.

S5. Pinned public schemas

Schema versions records the filenames, hashes, versions and counts at repository commit `18fa2cc768024b47789dc6fa243acd48478acac2`, dated 25 September 2026. All three files come from the public `stripe/openapi` repository. The commit message reads "Update OpenAPI for 2026-09-30.endive": the version strings name the API release the files describe, which postdates the commit.

The `private_preview` in file C's name does not indicate access to private systems. Stripe publishes these files under the MIT License. That license governs the schema-derived identifiers and locators in the event-type and signal-locator files; the package's third-party notice reproduces it.

File A is `latest/openapi.spec3.sdk.json`; B is `preview/openapi.spec3.sdk.json`; C is `preview/openapi.private_preview.spec3.sdk.json`. Paths are endpoint addresses. Resource schemas are component schemas with an `x-resourceId`; they describe kinds of objects. Some resource schemas in file C share an ID.

SDK file	Version	Paths	Resource schemas / distinct IDs	Event types
A	2026-09-30.endive	421	175 / 175	508
B	2026-09-30.preview	574	237 / 237	618
C	2026-09-30.preview	800	329 / 327	781

Event types lists the types declared under `x-stripeEvent` in each file. Each count includes 216 thin-event declarations whose names add `v1.` to a snapshot type in the same file. The counts do not also include the older `event.type` list. Signal locators identifies each counterpart's file and location using slash-separated field names and list positions. Slashes within endpoint names and media types, such as `application/json`, remain unchanged, so adjacent slashes can occur, as in `/paths//v1/webhook_endpoints`. The file's `locator` column therefore does not hold standard JSON Pointers, which would write each such slash as `~1`.

The signal-locator file lists only the 32 signals with partial counterparts. For the seven with none identified, the classification rests on a keyword search of all three files, so a counterpart under other terms is not ruled out. That search covered eight signals: SIG-03, SIG-34, SIG-35, SIG-38, SIG-39, SIG-40, SIG-56 and SIG-68. Its terms were `camera`, `emulator`, `inject`, `liveness`, `selfie`, `capture_session`, `device`, `tamper`, `manipulat`, `spoof`, `presentation`, `jailbreak`, `rooted`, `virtual`, `agent`, `tool_call`, `tool`, `mcp`, `llm`, `prompt`, `model_context`, `manifest`, `tool_version`, `hash`, `signature`, `signed_request`, `web_bot`, `bot`, `verified_agent`, `attestation`, `memory`, `skill`, `egress`, `allowlist`, `proxy`, `outbound_request`, `request_log`, `user_agent`, `ip_address`, `tls`, `ja3`, `fingerprint`, `http_status`, `api_error`, `request_id`, `api_request`, `allowance`, `spender`, `permit`, `signing`, `erc20`, `smart_contract`, `token_approval`, `approve` and `unlimited`. It searched event types, schema names, paths, keys, enum values and descriptions. For SIG-03, identity-verification outcomes such as `selfie_manipulated` and the `require_live_capture` setting record no device, camera, emulator or injection context. For SIG-34, the `agent_details` fields name the agent behind a payment or token and file C has MCP keys for claimable sandboxes, but neither records a tool call or its input provenance. For SIG-40, an API key's `ip_allowlist` controls inbound use of the key, not egress. For SIG-56, the event envelope's request ID and idempotency key, an API error's `request_log_url` and aggregate API-error health alerts give no per-request context. For SIG-38, the search found the `agent_details` fields, which the signal-locator file records as its partial counterpart.

Every listed counterpart is partial. An event envelope (the wrapper around an event) supplies only some required provenance. A request ID does not supply request context, and a score does not establish the behavior it scores. Likewise, current state does not supply a complete history. Onramp-session objects (sessions for buying cryptocurrency) and event-destination configurations (settings for event delivery) illustrate this gap. The pinned schemas record part of their state without complete matching event streams. Stripe's crypto documentation describes a webhook for each onramp-session status change after creation, but none on creation. Neither the event types that the three files declare under `x-stripeEvent` nor their lists of webhook-endpoint events name a crypto or onramp event.

S6. Hypothetical investigations and reproducible arithmetic

Table S6.1 summarizes the two investigations. The synthetic cohort contains 400 completed transactions, 100 in each of four non-overlapping account cohorts. Each cohort fixes one account's transactions using the same definition and outcome window. These synthetic rows make the counting rules reproducible. They contain no report-arrival dates, so the ratios assume that every report on the four cohorts has arrived. They cannot show reporting delay.

Once each report is linked to its transaction, each of the four accounts, A to D, has two distinct transactions with chargebacks, so its chargeback ratio is `2/100 = 2%`. Pooling the same measure across the four accounts gives `8/400 = 2%`.

Consider non-overlapping cohorts with the same outcome definition and outcome window. Let $T_i > 0$ be account i's transaction count and D_i its count of distinct transactions with chargebacks. Then

$$r = \frac{\sum_i D_i}{\sum_i T_i} = \sum_i w_i r_i, \qquad r_i = \frac{D_i}{T_i}, \qquad w_i = \frac{T_i}{\sum_j T_j}.$$

The weights are nonnegative and sum to one, so the pooled ratio lies between the smallest and largest account ratios.

Fourteen transactions have a resolved pre-dispute alert, including two of the eight transactions with chargebacks. Counting each transaction once gives `8 + 14 - 2 = 20`. The separate chargeback-or-alert-resolution measure is therefore `20/400 = 5%`. Adding all flags would count two transactions twice and wrongly give `22/400 = 5.5%`. This broader measure is neither a fraud rate nor the chargeback ratio above. The example assumes no dispute-monitoring threshold for either measure. The broader measure is `8/100` for A and B, and `2/100` for C and D; the pooled 5 percent lies between them.

The chargebacks on D001 and D002 are recorded at account-X, whose own transactions lie outside all four cohorts. Both transactions remain in D's cohort and count once; account-X adds no transactions to the denominator. Counting reports where recorded shows D at `0/100` and assigns both reports to account-X. Linking reports to transactions restores D to `2/100`, the same as the other accounts. The mismatch needs explanation but does not prove concealment. Legitimate merchant-of-record or facilitator arrangements can also record disputes on another account.

Both measures exclude reports on transactions outside the four cohorts. A missing transaction link leaves a measure that depends on it incomplete. Any change to report links or outcome definitions must be stated explicitly. If it applies alike to every cohort, the weighted-average formula still holds. The exact 95 percent intervals in manuscript Section 8.2 are Clopper-Pearson intervals, and its p-value comes from Fisher's two-sided exact test of 8/100 against 2/100.

Trace A uses a separate hypothetical timeline (manuscript Sections 8.1 and 8.3). The agent requests seven days of history before the key's exposure; only the final two hours are available. A detection already running on the platform freezes its first compromise assessment at +30 minutes after publication of the key. The destination-reuse link becomes known at +1 day, although it was effective before exposure. The +30-minute assessment cannot use that link, and the missing history leaves the onset of any misuse inconclusive.

Table S6.1. The two investigations at a glance. Every observation is hypothetical. Manuscript Sections 8.1 and 8.2 identify the supporting research and how far its findings transfer to this setting.

Element	Trace A: exposure and possible payout redirection	Trace B: storefront links and possible dispute evasion
Trigger	An external observation of an exposed key	A platform risk assessment and a rejected application
First questions	Did use change after exposure? What requests and account changes followed?	Which storefronts share templates, descriptors or merchant identifiers?
Main analysis	Join events by time; examine the history and reuse of payout destinations	Examine hard and soft links, and outcomes linked to each transaction, across accounts
Benign alternatives	Developer testing from a new network or an internal scanner; a business with several legitimate accounts, or an intermediary receiving funds for several clients	A legitimate multi-brand business, shared template vendor or aggregator
Evidence limit	Payout redirection is reasoned about by analogy with payroll diversion; timing premises come from Microsoft's report of follow-on business email compromise payment fraud soon after session theft and a deliberate key-exposure experiment	Complaint allegations are not findings; resolving pre-dispute alerts can change chargeback ratios
What limited evidence cannot show	A partial baseline cannot show that there was no earlier misuse	Neither soft links nor a hard link from a verified identifier by itself establishes common control, intent or who is behind the activity
Proposed outputs	Hunting candidate; detection candidate with an evaluation plan; recommendations to consider rotating or revoking the credential and a payout pause or hold; intelligence draft	Cluster intelligence assessment; recommendation to review the newly onboarded merchant accounts linked to the storefronts; detection candidate built on hard links between storefronts and accounts
Who acts	Authorized humans or accountable operational systems decide. The urgent exception covers a reversible payout pause or hold; rotation or revocation follows normal review	Authorized humans or accountable operational systems decide whether to reject or terminate accounts; the agent does not

CHECK_CANDIDATE.py checks these calculations and excludes the +1-day destination-reuse link from the +30-minute assessment. It also checks identifiers, counts, feature lineage, citation-reference agreement and links between files. REPRODUCE.md gives build and verification instructions. These are checks of the supplied artifacts and arithmetic, not an evaluation of the proposed system.

S7. Evaluation conditions

These conditions explain the scope and comparisons behind the tests in S7.1. E8's release-gate test appears there alongside the other eight; S8 adds practitioner judgments of availability and usefulness.

E1. Do coverage and learned-time records reduce unsupported conclusions? The confirmatory test in S7.1 concerns cases with incomplete history or links learned late, with a stratum of cases with complete history for its non-inferiority measure. A broader timing study would compare minute-level joins with coarser timing across a realistic incident population. It would measure exposure-to-use and session-issuance-to-use intervals, detection or triage gains, collection costs and false alarms from benign changes. Microsoft's report of payment fraud attempts soon after session theft and the deliberate key-exposure experiment motivate precise timing; neither establishes its typical benefit.

E2. When can missing outcomes be estimated? Define the outcome channel, population and assumptions before selecting an estimator. Simulation can test behavior under known assumptions, including cases without overlap, where no comparable cases received another decision. It cannot verify those assumptions or recover unknown real outcomes. Later observational comparisons must separate populations with supported overlap from those whose outcomes cannot be estimated. Test sensitivity to late, mistaken and deliberately false reports. Allowing attempts that a policy would otherwise block requires separate ethical, human and operational authorization. The motivating theoretical preprint proposes methods; this paper claims no recovery of missing outcomes on platform data [2].

E3. Do report linkage and cluster monitoring reveal evasion earlier? Compare the network's monthly rule, its ratio, its minimum count and any minimum value, applied to each account as reports are recorded, with the same rule applied to each cluster after linking reports to transactions. In S7.1, each mention of the network's ratio for E3 means this complete rule, including its minimum count and any minimum value. A third arm applies the rule to each account after linking, separating linkage from clustering. When every account has captured payments in the denominator period, a pooled ratio cannot exceed the highest account ratio, so a lead from clustering can come only from a condition on totals, such as the minimum count or, where the network sets one, a minimum value. Report separately any cluster flag that depends on reports received by an account with no captured payments in that period, such as a closed account or, where the denominator is the previous month's captured payments, an account in its first month. Keep the eligible transactions, outcome definition and reporting periods consistent. Apply the account arms to platform and connected accounts and, where a network monitors a different unit, such as Visa's static statement descriptor and acquiring bank, also to that unit. Evaluate broader measures, including chargeback-or-alert-resolution, separately. Apply each program's exclusions as the network's documentation states them, such as VAMP's exclusion of fraud reports that qualify for Compelling Evidence 3.0. State whose report view each arm uses: the programs also count disputes that the provider handles on a business's behalf and that do not appear in that business's data [3]. Report the share of reports linked to transactions and how duplicate reports were removed. Measure false flags on legitimate multi-brand businesses. Report erroneous links between unrelated accounts sharing IP addresses, such as through a common proxy, as a secondary measure; S7.1's failure criteria do not include it [4].

E4. Can the records measure harm to legitimate users? The feasibility test in S7.1 asks whether control changes have computable harm rates and intervals. It precedes E1 to E3 and E9. A broader evaluation would compare detection gains with failed attempts and other costs to

legitimate users. The identity-verification and login-challenge studies report user outcomes under their own procedures [5, 6]. Their rates cannot determine whether a proposed payment control is worthwhile.

E5. Does the agent improve the investigation? Compare the governed agent with well-equipped human investigators and suitable decision rules, including simple thresholds where appropriate. Sharma's agent underperformed its underlying classifier's threshold on 60 synthetic cases [7]. That limited result motivates testing the agent's added value. Give each approach the same information, frozen at an as-of cutoff. Report case base rates and judge outcomes without knowing which approach produced each finding. Set error, abstention and escalation rules in advance. Measure correct findings, errors, abstentions, human review burden, cost and latency, and include adaptive adversaries. The study records reviewer time for investigation, tool or agent operation, checking, release and rework on the same basis in both human arms. The storage contract holds these times only for work recorded in the agent-and-approval ledger.

E6. Do the agent's safeguards hold under adaptive attack? Test prompt injection, approval rebinding (reusing approval for an altered operation), memory poisoning, limits on outbound data, planted omissions of retrieved records that count against a claim, and steering the agent away from queries that would return such records. Let adversaries adapt to the defense. Measure security failures and the effect of safeguards on investigations completed. Prior adaptive attacks motivate this test but do not establish that these safeguards work [8].

E7. Can lessons be reused without spreading mistakes? Measure time to useful reuse, negative transfer (a lesson degrading later investigations) and poisoning. Vary validation strictness, release scope and withdrawal conditions. General-purpose agent research supplies methods and evidence of failures, not payment-platform results [9, 10].

E9. Do channel states reduce bias relative to the platform's current labels? Use decisions the platform already logs, with their disputes, early fraud warnings, returns and other reports. For a fixed cohort, build labels once under the platform's current label rule, including its maturity period and its treatment of blocked or reversed items, and once under the channel-state and maturity rules (manuscript Section 5.3). If the platform has no such rule, use for each rail one maturity period equal to that rail's standard window, with blocked attempts excluded. Compare the resulting labels, an existing control's measured error rates and the ranking of two candidate rules. On decisions with a randomized holdout, compare the bias of an evaluation quantity that the platform computes today under both readings against the holdout's outcomes after the longest reporting window has closed, with report truth reviewed on a random sample. Only that bias comparison counts toward E9's pass rule. Report, by channel, how often the records determine an affected item's channel state, clock start and window rather than recording them as unknown. Record and report any rule event that reopens a window during follow-up. The motivating theoretical preprint models reporting and maturity for one channel [2]; E9 asks whether recording several channels' states matters on platform data.

Record the provenance of synthetic, replayed and emulated data when created. Agent-driven adversary emulation would also require the authorization and safeguards in manuscript Section 6.5; this paper performs and authorizes none.

S7.1 Pre-specified hypotheses and criteria

For each question in manuscript Section 9.1, this section proposes a hypothesis, data design, primary measure and result that would count against the specification. E2's interval settings remain

deployment-specific choices, to be fixed before any data are examined. Where a question in Section 9.1 is broader, the test below addresses the part that bears on the three record requirements or, for E5, on the agent's added value. Thresholds and sample sizes are proposed defaults. E1's non-inferiority margin and E8's tolerable and assumed release rates are named parameters that a platform fixes and records before data collection, and E9's negligible bias reduction is one that it fixes before any data are examined; this section does not set them. A platform may change them before it starts, if it records the change before examining any data. No data have been collected, and whoever runs the plan can register it with a time stamp.

Four rules apply to all nine. First, each evaluation freezes the information each method may use at an as-of cutoff and separates development from test data. Second, it states how true outcomes are decided, by whom, and blind to which method produced each finding. Its error and abstention rules are set in advance, and it states base rates. Third, where ground truth comes from labels shaped by control decisions, it says how that bias is avoided. Otherwise it reports relative performance among detected cases only. Fourth, records from tests, replays and emulations are marked when created and kept out of production labels and lessons.

Each evaluation is judged separately, without correction for multiple tests, because support requires every confirmatory evaluation to pass. Each confirmatory evaluation targets 93 percent power: the chance of passing its complete rule under stated effect assumptions. If each reaches that target, all four pass with probability at least 72 percent. Sizing must account for every pass criterion and case or reviewer clustering. The minimum counts below are not power calculations; the sizing assumptions must be recorded before main-study data are collected. E1, E3, E8 and E9 are confirmatory and the rest are exploratory. The specification is not supported in a setting if any confirmatory evaluation fails. A negative result removes support for a requirement, or for its cost, in that setting. It does not show that the underlying idea is wrong. Investigators cannot be blind to the added records, so the raters, not the investigators, are blind.

E1 (tests the second requirement; confirmatory). Hypothesis: coverage and learned-time records reduce the share of unsupported conclusions on cases with partial history or with links learned late, without reducing the correct, supported conclusions reached per case.

Data and design: closed real cases of that kind, each worked by different investigators with and without those records, and rated by two blind raters. Investigators are assigned to the two conditions at random, balanced so that each works cases in both conditions and none works the same case twice. Raters score conclusions as single propositions under a rubric fixed in advance: they split compound conclusions into their propositions and merge restatements of one proposition, so that each distinct proposition is scored once however it is phrased or grouped. A stratum of closed cases with complete history is worked and rated the same way. Who investigates is fixed before the study starts: people, the agent, or both as separate arms, each analyzed on its own. If both arms are run, E1 fails if either arm meets a failure criterion, and the power target applies to both arms together. Conclusions reach the raters in a common format without coverage or learned-time field names. Each rater records which arm they believe produced each conclusion, so that blinding can be checked. A 30-case pilot, excluded from the main study, estimates the level without the records for sizing. The main study will be sized for 93 percent power for its complete pass rule, assuming the share truly falls by half and correct, supported conclusions per case do not fall. Intervals allow for several conclusions per case.

Primary measures: the share of conclusions rated unsupported by the evidence then available and, co-primary, the number of correct, supported conclusions per case, rated by the same blind

raters against the closed case's resolution. Abstentions per case are reported for each arm and stratum.

Failure criteria: Fails if, in the cases with partial history or links learned late, the share falls by less than a third of the level without the records, or the 95 percent interval on the paired difference includes zero. It also fails if, in those cases or in the complete-history stratum, the lower 95 percent bound on the difference in correct, supported conclusions per case (with the records minus without) lies below minus the non-inferiority margin.

E2 (tests the first requirement; exploratory). Hypothesis: the control-decision record is complete enough to estimate outcomes under other decisions (manuscript Section 7.1).

Data and design: a simulation with known effects first, then the last quarter of decisions where the platform already randomizes traffic under its own authorization. Record that quarter's start and end dates before examining the data. The target is matured reported outcomes: holdout rates count the reports received once each channel's window has closed, and report truth is reviewed on a random sample. A usable probability is one logged with the feature values and model version that produced it. The deployment chooses the intended coverage defined under the primary measures, but it must be at least 95 percent, because the failure criteria allow no more than 10 percent of overlap populations outside their intervals.

Primary measures: the share of randomized decisions logged with a usable probability, feature values and model version. Also, the share of overlap populations whose holdout rates fall outside the predicted interval. Each interval predicts the observed outcome rate in a held-out population, including its sampling variation. Before examining any data, specify the estimator, interval construction, intended coverage and overlap populations. Intended coverage is the planned share of repeated holdout rates the intervals contain. These deployment-specific settings are not selected here.

Failure criteria: Fails if fewer than 95 percent of randomized decisions carry a usable probability, or more than 10 percent of overlap populations fall outside the predicted interval.

E3 (tests report linkage, which the first requirement depends on, together with cluster monitoring; confirmatory). Hypothesis: linking each report to its transaction, and computing the network's monthly ratio for each linked cluster, flags evasion earlier than the network's ratio for each account.

Data and design: at least 30 linked-account clusters with later confirmed outcomes, channels kept apart, and at least 100 legitimate multi-brand businesses from onboarding. The pass rule is judged only on clusters selected by a confirmation route documented as independent of either monitoring rule; clusters from other routes are reported separately. Adjudicators blind to both rules' outputs fix the onset of evasion for each cluster from case evidence. Each program is applied as the network's own documentation states it, with the provider's guide used only as a secondary summary. Time to flag is analyzed as paired, right-censored time-to-event data: a cluster that a method has not flagged by the end of follow-up is censored there for that method. A third arm computes the network's ratio for each account after linking. It separates the effect of linking from that of pooling accounts into a cluster. The failing result applies to the stated contrast, and the third arm attributes any lead.

Primary measures: days from the adjudicated onset of evasion to the flag, and false flags. The lead time is the difference in restricted mean time to flag, account arm minus cluster arm, up to a horizon fixed before any data are examined.

Failure criteria: Fails if the lower bound of the 95 percent interval on the lead time, estimated from the paired censored analysis, is zero or less. It also fails if the share of legitimate multi-brand businesses flagged is more than 5 percentage points above the share under the network's ratio for each account.

E4 (tests the first requirement; exploratory). Hypothesis: for each control change (a new rule, a threshold move or a model version), the records support a rate of harm to legitimate users with a defined denominator.

Data and design: a sample of control changes, run before E1 to E3 and E9 as a feasibility check.

Primary measure: the share of sampled changes with a computable rate and interval.

Failure criteria: Fails if fewer than 80 percent of sampled changes have one.

E5 (tests the agent's added value; exploratory). Hypothesis: the agent with human review matches or improves confirmed findings per reviewer hour compared with a person using tools. Case error rates are no more than 2 percentage points higher than either comparator arm's.

Data and design: at least 700 frozen cases and three arms: threshold rules, a person with tools, and a person with the agent. Errors are counted per case. Both human arms see the same records. In E5, reviewer time covers human investigation, tool or agent operation, checking, release and rework. Measure these activities on the same basis in both human arms. Include work on cases that produce no correct finding.

Primary measures: correct findings, errors, abstentions, reviewer minutes per correct finding, cost and latency.

Failure criteria: Fails if errors in the agent arm exceed those of the human arm or the threshold-rules arm by more than 2 percentage points, or reviewer minutes per correct finding are higher. For minutes, higher means that the 95 percent interval on the agent arm's ratio to the human arm lies wholly above one. Avoiding a failure criterion does not by itself establish equal error rates or improved efficiency.

E6 (tests the third requirement; exploratory). Hypothesis: the safeguards prevent attacker-influenced content from gaining unauthorized control over tool calls, outbound traffic, released claims and the evidentiary status of records.

Data and design: a red team that adapts to the defenses from the four attacker positions in manuscript Section 6.5, each with its attempts, budget and model access stated. Content is planted in every input the agent reads, and every field the agent can write is checked for it. Planted content includes attempts to make the agent omit retrieved records that count against a claim and attempts to steer it away from queries that would return such records.

Primary measures, per position: attempts that cause an unauthorized tool call, transfer or memory write; released material claims that a blind adjudicator judges false; and missing links back to agent outputs.

Failure criteria: Fails if any write to released organizational memory occurs without human release or any required link back is missing. It also fails if a false claim about a named entity is released, either clearing it or stating that activity is absent. Success for each position is reported with its budget, without a pass mark.

For these criteria, a write is one that adds or changes a released claim; monitoring, expiry and withdrawal records are status records. Against the insider position, a release or link removal by that insider is an authorized action. E6 reports whether the release record, the logged link removal, expiry and correction detect and limit it, and the failure criteria apply to releases and removals that position did not make.

E7 (tests the third requirement; exploratory). Hypothesis: re-testing, scoped release and withdrawal keep a reused lesson from degrading later investigations without delaying useful lessons.

Data and design: lessons released under varied strictness, planted poisoned lessons, and an expiry schedule disclosed to the red team.

Primary measures: the rate of degraded later investigations, the time to reuse, and the planted lessons released and still active.

Failure criteria: Fails if a planted poisoned lesson is released and still active after its expiry or re-test date. It also fails if the median time to reuse exceeds 14 days.

E8 (tests the third requirement; confirmatory). Hypothesis: reviewers and releasers catch planted wrong drafts when cited records are checked at the rate and materiality tiers planned for deployment and they do not know which drafts are planted. Claims about named entities are checked in full, and a person releases every draft.

Data and design: planted drafts, interleaved with production drafts at realistic volume and at a stated prevalence close to the expected production error rate, spread across reviewers, with intervals that allow for clustering by reviewer. The realistic-prevalence arm must reach the required numbers of planted drafts and planted false exonerations by itself, because both failure criteria apply to it. A higher-prevalence arm may be added only for secondary estimates by error type; it does not count toward the failure criteria, and both arms are reported. Reviewers and releasers are not told which drafts are planted. Planted drafts concern synthetic entities or closed cases whose cited records are test-marked and whose named entities have no account action pending. They carry the test environment mark when created; reviewer-facing views hide it, while the stores enforce their exclusion from production labels, intelligence and lessons. The gateway refuses any control request or urgent-containment routing that originates from a test-marked draft, and it tells the decider that the draft is planted before any action. Released planted drafts are withdrawn after scoring. Planted errors include false exonerations, drafts that wrongly clear a named entity; omissions of retrieved records that count against a claim, including records that the checker's fixed rule marks and records it leaves to the sample (manuscript Section 6.4); and drafts whose only support for a material claim is a released item or a record linked to one. The mix of planted error types is fixed before the study starts. Some planted errors come from the agent's own earlier runs and the rest are written independently. The number of planted false exonerations is sized for 93 percent power to show a release rate below the tolerable rate, given an assumed true release rate stated in advance, and the number of planted drafts is sized so that the complete rule, with the catch-rate criterion, reaches that power. With none released, showing a rate below 1 percent at one-sided 95 percent confidence needs at least 299. If the true rate is 0.1 percent, 93 percent power to show a rate below 1 percent needs 628. If planted false exonerations made up 5 percent of the drafts reaching review, 628 of them would take 12,560 drafts, about 628 days at the 20 drafts a day assumed in manuscript Section 6.4. The plan states the tolerable rate, the assumed rate, the prevalence and the implied duration together.

Primary measures: the catch rate, overall and by error type (false exonerations, other wrong claims, omissions of retrieved contrary records, and support resting only on released items or records linked to them); the release rate of planted false exonerations; reviewer minutes per released draft; and how often the recorded random sample of unmarked uncited returned records contained a record that counted against the claim. For the catch rate by reviewers and the releaser, a planted error counts as caught only if one of them identifies it, by naming the wrong claim or the missing record; a draft returned or left to expire without that finding does not count. The checker's catch rate counts planted errors that its fixed rules reject. Catch rates are reported for the checker, reviewers and releaser, for each prevalence arm, and for errors from the agent's own runs and independently written errors.

Failure criteria: Fails if the lower 95 percent bound on the catch rate by reviewers and the releaser is below 80 percent, counting planted errors that the rule checker did not reject, or the one-sided 95 percent upper bound on the release rate of planted false exonerations exceeds the tolerable rate stated in advance. Both criteria apply to the realistic-prevalence arm alone.

The release gate has a cost that E8 measures. If a system produced 20 drafts a day and a person needed 30 minutes to read each draft's cited records, review would take 10 hours a day. The gate therefore moves analyst work and may not remove it. At realistic volume the reviewer may need to read a sample of each draft's cited records, tiered by materiality. Claims about named entities are always checked in full, and a person still releases every draft.

E9 (tests the first requirement's channel states; confirmatory). Hypothesis: compared with the platform's current label rule, channel-state and maturity records reduce the bias of an evaluation quantity that the platform computes today.

Data and design: two parts with one comparator. The comparator is the platform's current label rule, with its existing maturity period and its treatment of blocked or reversed items, recorded before any data are examined. If the platform has no such rule, the comparator uses for each rail one maturity period equal to that rail's standard window, with blocked attempts excluded. (a) Historical reclassification: a fixed cohort of payment attempts, payouts and account decisions, with the decision logs, disputes, early fraud warnings, returns and other reports the platform already holds. Labels are built twice, once under the comparator and once under C1, which reads a missing report as a provisional negative only where its channel matured open under its original meaning. The cohort, channels and read times are recorded before any data are examined. (b) Bias against a randomized holdout: decisions where the platform already randomizes traffic under its own authorization, as in E2. At operational read times, an evaluation quantity that the platform computes today, such as a detection's precision or a control's false-positive rate, is computed under both readings. Before any data are examined, the plan states how items that either rule leaves unlabeled enter the quantity, whether excluded, bounded or estimated under a stated delay model, and how channels without a defined reporting window enter the holdout comparison. The quantity, its population, denominator and read times, and the method applied to unlabeled items are the same under both readings, so that the comparison tests the records rather than a change of estimator. Each reading is compared with the holdout's outcomes after the longest defined reporting window has closed, with report truth reviewed on a random sample. Rule events that reopen a window during follow-up are recorded and reported. The main study will be sized for 93 percent power for part (b), which carries the pass rule, under stated effect assumptions.

Primary measures: (b) the reduction in absolute bias of the stated quantity under C1, relative to the comparator, with its 95 percent interval. Reported but not counted as support: (a) the share

of the comparator's negatives that C1 classes as closed, delayed, changed in meaning, immature or without a defined window. Items whose channel state is unknown are excluded from that share and reported separately as a coverage shortfall, with, by channel, the share of affected items whose channel state, clock start and window can be determined rather than recorded as unknown. Also reported: the reports that later arrive on the comparator's negatives, and the change in an existing control's measured error rates and in the ranking of two candidate rules.

Failure criteria: Fails if the lower 95 percent bound on the bias reduction is at or below the negligible bias reduction, a named parameter fixed before any data are examined. Measure (a) does not count toward passing, because the share that C1 reclassifies follows partly from its definitions and from the cohort's composition.

S8. Practitioner review protocol (not yet run)

This proposed protocol assesses whether the specification fits investigative practice. It measures availability and perceived usefulness, not detection performance. Recruit six to ten fraud, abuse, risk, trust-and-safety, data-engineering or privacy practitioners from at least three organizations, with consent and required institutional ethics review.

Give each participant the signal inventory (S3), responsibilities (S4) and traces (S6), with and without the coverage, learned-time, decision-link and release records. For each signal, ask whether it is collected today (yes, partly, no), its typical retention and its freshness (at decision time, in minutes, daily). Also ask about usefulness in recent cases and the cost to an adversary of evading it. For each responsibility, ask whether it exists and which role owns it. For each trace, record the conclusions the participant would draw in each condition.

Participants may also review a small set of planted wrong drafts as a pilot for E8; such a pilot cannot reach E8's realistic volume and does not test its criteria. Publish the protocol with ratings aggregated across organizations and with free-text quotes stripped of identifying detail, identifying signals rated missing, redundant or infeasible. No published rating or quote may show which organization lacks a signal, because that would disclose a control gap. No participant data have been collected.

S9. Extended comparison with prior work

Manuscript Section 2 develops the closest precedents for C1 to C3. This section places them within research on payment decisions, investigation and agent memory. The comparisons concern the cited designs and findings, not every capability their systems might support. S1 records source versions, publication status and supporting passages. Table S9.1 summarizes the nearest precedents for each requirement and what the specification adds to them; the subsections that follow give the wider context.

S9.1 Payment decisions and the outcomes they reveal

Payment engineering already connects records, investigation and protective decisions. Stripe describes similarity clustering for investigators [43]. It also describes a retraining cycle for card-testing models: labels are derived partly through manual expert review, and retrained models are evaluated on offline data and compared with the old model in blue-green tests before deployment [44]. These vendor accounts establish publicly described practices in their stated settings. They do not independently measure the effectiveness of the proposed specification.

Outcome-aware evaluation also predates this work. Manapat's 2015 talk abstract describes adding randomness to production fraud scoring so that a blocking model's precision and recall can be estimated from production data [21]. Adyen evaluates suggested payment interventions using transactions logged under an existing policy [16]. A Meta preprint proposes estimators for blocking's false-positive rate, which cannot be monitored directly after a user is blocked, and tests them on simulated blocking policies [17]. These approaches motivate retaining the decision and the conditions under which its outcome became observable. They do not make an unreported outcome known.

Two recent proposals connect evaluation evidence to operational authority. Deng treats fraud automation as an authorization decision because its evaluation labels are selective and delayed [18]. Solozobov specifies decision-linked records for facts learned later, including the arrival of ground truth [19]. C1 applies that linkage to several outcome channels. It records what a control closed, delayed or changed in each channel, alongside reports and assessments of their truth. Its contribution is this payment-investigation record contract, not a new estimator of missing outcomes.

Table S9.1. The three requirements and their nearest precedents. The middle column states what the inspected sources establish; the right column states what the specification adds for cross-product investigation on a payment platform. The search was not systematic (manuscript Section 9.2).

Requirement	Nearest precedents and what they establish	What the specification adds
C1: outcomes linked to decisions	Selective-labels evaluation assumes data in which each case carries the decision taken [11]; learning from logged decisions treats the decision and its probability as required data [12]; decision provenance records the data flows around a decision [13]. In consumer credit, reject inference faces the same problem for rejected applicants; a review finds the common methods' assumptions mostly implicit and no method uniformly better [14]. Dhama's missing-data model carries, for each transaction, the authorization decision, whether fraud was reported and whether that report had arrived by training time, for one reporting channel, and models label corruption separately [2]; a second theoretical preprint by the same author bounds learning under delayed, censored, corrupted and counterfactually missing feedback, treating each as a distinct impairment [15]. Adyen evaluates payment-time interventions offline from transactions logged under the existing policy [16]; a Meta preprint proposes estimators for the false positive rate of blocking, which cannot be monitored directly once a user is blocked, and tests them on simulated blocking policies [17]; a fraud-operations preprint treats automating an action as an authorization decision because its labels are selective and delayed [18]; a decision-event schema records facts learned after a decision, such as when ground truth arrived, as linked records that reference the decision [19]; the coverage model in the C2 row also links records to decisions and distinguishes refused execution branches from late reports [20]; a 2015 talk abstract describes adding randomness to production fraud scoring so that a blocking model can be evaluated, and new models trained, although blocked transactions' outcomes are not observed [21].	Several outcome channels per decision, each with its own clock-start event and reporting window; a record of what each decision closed, delayed or changed in each channel, and a status for each channel at each read time; separate fields for delay, unobservability and report truth, so the evaluation states joint assumptions; the recorded policy probability, inputs and case assignment that those assumptions need (manuscript Section 7.1). A platform that already joins decision logs to dispute records can use C1 to specify what the join must preserve.

Table S9.1 (continued).

Requirement	Nearest precedents and what they establish	What the specification adds
C2: query coverage and learned time	Temporal databases separate valid time from transaction time [22]; completeness of a query answer can be derived from statements about which parts of the data are complete [23]; the Dataflow model assumes a system processing unbounded data never knows whether all data have arrived, treats watermarks as heuristic estimates of progress, and revises results when late data arrive [24]. Bitemporal agent memories keep both times [25–27], and Quipu gates writes to a knowledge graph while retaining validity and recording times, with some policies requiring human approval bound to the submitted evidence [28]. A preprint audit of sixteen deployed agent frameworks reports none that writes a run record a reader can check without trusting the writer; a language model scored the audit grid, and agreement with human coders fell below the registered threshold. It proposes a log kept outside the agent's harness [29]. A coverage model separates event time from verifier receipt time, judges whether a verifier's evidence was complete at a stated cutoff, links records to decisions and distinguishes refused execution branches from late reports; it also separates absence from one retrieval from absence in a verifier view whose closure, including retrieval coverage and retention, is justified, and specifies that basis without designing its implementation [20].	The gateway outside the agent attaches a coverage record to every result: the entities and interval requested and evaluated, the as-of cutoff, each source's event and ingestion watermarks and known gaps, the retention horizon, limits reached, access filtering, sampling and schema versions. Fixed rules outside the agent derive whether a result is complete, partial, truncated, denied or unavailable, and a scope-limited flag confines absence claims to the authorized scope (manuscript Section 6.2). Every fact, including a derived record, keeps event time and the time it first became available to queries (S4.1).
C3: human release and dependence	Truth-maintenance systems record each belief's justifications and retract conclusions whose support changes [30, 31]; STIX keeps observed data apart from intelligence assertions and opinions [32], and US intelligence analytic standards ask analytic products to distinguish underlying information from assumptions and judgments [33]. Anamnesis starts model-derived claims as candidates, and no memory operation can raise their authority without a separately recorded grant; the grantor need not be a human [27]. GovMem groups dependent agent traces when estimating support instead of counting them as independent votes, permits automatic promotion and routes unresolved candidates to human review [34]. A data-governance preprint admits only steward-verified records as precedent, which then serves as evidence (S9.4) [35]. Albarrak and Jagtap place human approval before intelligence is shared, but do not state that analyses kept in their institutional-memory store pass that gate [36]. MAP-Graph writes memories without human release (S9.3) [37]; Li and coauthors check an authorization state that need not come from a person [38]. A formal study distinguishes independent observations from reports derived from shared evidence and requires a dependence model as well as ancestry [39]; a threat-detection design drops its own compiled detections at ingest (S9.4) [40]; an engineering assurance design binds evidence to the exact artifact, and its governing profile can require human authority [41]. AgenticCyOps provides analyst oversight without a human release decision for every item before organizational reuse [42]. TOKI's optional external callback, which may be a human, only chooses between contradicting facts [25].	The recorded grant is specialized to a human principal for every conclusion entering organizational memory, as the data-governance gate does for precedent, and released items stay context: a released conclusion is never counted as a new or independent observation, and a claim about current conduct must cite observations of that conduct from the current period, not scores or control decisions alone (manuscript Sections 4.2 and 6.4). Labels, assessments, control decisions, detections and threat-knowledge items carry a link to every agent output and every released organizational-memory item open in the writer's case or session. The labeling, assessment, control, detection and threat-knowledge interfaces, rather than the person, write these links automatically, so they also cover records a person writes with an agent output open in the case. Linked records count as dependent; a missing link does not establish independence (manuscript Section 4.2). Responsibilities 21 and 22, quality review and human release, are inherited from threat-intelligence work (S4).

S9.2 Intelligence production and investigation

STIX is a structured language for representing and exchanging cyber threat intelligence [32]. Such representations make analysis reusable. STIX already separates observed data from intelligence assertions and opinions, but it does not address how opinions should be interpreted [32]. C3 supplies a rule for what a reused conclusion can establish in this specification.

ExCyTIn-Bench evaluates agents on multi-step questions over security logs [45]. This benchmark establishes relevant investigative tasks and a way to evaluate particular capabilities. The present specification adds requirements for interpreting the inputs to such tasks and preserving the status of their outputs across payment products.

PROVSEEK is especially close to the requirement that a claim remain checkable. Its agent cites audit-event identifiers for claims and reports searched-for artifacts it did not find [46]. C2 additionally requires a gateway outside the agent to record query coverage. A missing artifact then has an explicit search scope, rather than relying on the agent's description of what it searched.

S9.3 Permission to act and authority to reuse

Recent proposals bind authority to particular artifacts or actions. MAP-Graph represents memories, claims and actions in a provenance graph and records each action-gate decision with its supporting memories; the gate can return an AskUser decision for some high-risk actions [37]. Li and colleagues relate an execution attempt and its artifact to a downstream-use authorization state, checked at publication [38]. Agile-V binds evidence to an exact artifact and links effects to authorization records; its governing profile can require human authority [41]. These are concrete precedents for authorization records. C3 makes a narrower, stronger choice for organizational memory: a person releases every conclusion, and the released conclusion remains context or hypothesis.

A survey of persistent agents calls for records tying actions to the state that justified them [47]. EDDOps links versioned evaluation records to the agent changes they prompt [48]. Both motivate preserving the connection between an output and its later effects. Under C3, labels, assessments, controls, detections and threat-knowledge items link back to every agent output and every released organizational-memory item open in the writer's case or session. Human release does not erase this dependence or turn the output into independent evidence.

S9.4 Memory, dependence and revision

Agents can learn reusable lessons from experience. ExpeL extracts lessons, which it calls insights, from its successes and failures on training tasks, adds them to the prompt for later tasks and retrieves similar successful trajectories as examples [49]. This capability explains the value of reuse, but reuse can propagate an earlier mistake. A-MemGuard checks memory-derived reasoning during use, consulting lessons from stored failures [50]. Manuscript Section 7 proposes testing new lessons against counterexamples before reuse, then testing their effects in E7. This proposed control has no measured payment-platform benefit here.

The choice of memory structure depends on purpose. An investigative system needs to know whether a retrieved item is an observation or a conclusion. Studies of memory consolidation show that it can erase a claim's source; external source records restore much of the lost control in their test settings [51, 52]. Those settings used synthetic customer-service histories and benchmark attacks, not payment investigations.

SSGM separates an append-only observation log from derived memory and checks updates against protected facts [53]. A shared-memory service records writers, derivation links and supersession; its authors evaluate it without a baseline [54]. PROV-AGENT extends provenance records to prompts, responses and decisions, linking them to downstream outcomes [55]. These precedents support typed records and traceable revision. C3 does not claim to invent them.

Source ancestry is necessary but not sufficient to establish independent support. A formal treatment of evidence dependence distinguishes independent observations from reports derived from shared evidence [39]. C3 follows the same concern: several agents repeating one report do not create several independent observations. A released conclusion can guide the next query, but evidence for that conclusion must remain traceable to the underlying records.

Two recent designs bear directly on C3. A data-governance preprint commits no agent-proposed metadata decision before a steward reviews it. It proposes a promotion gate that admits only verified records as precedent, records each promotion and lets a reviewer demote a record, flagging the decisions that relied on it; the verified precedents then serve as evidence for later proposals [35]. A threat-detection design tags every detection compiled from one of its hypotheses and drops tagged observations at ingest, so its own conclusions cannot re-enter as fresh evidence. It leaves as a residual risk an analyst who writes such an alert into a report that is later ingested, and its reference implementation does not yet exercise the rule [40]. C3 keeps released conclusions as context rather than evidence, and its interface-written links cover records a person writes with an agent output open in the case.

S9.5 Time and coverage

Timing matters when it changes which information a decision could use. Dataflow addresses out-of-order streams, where the system cannot assume that all events have arrived [24]. Leakage research identifies information about a target that should not legitimately be available to a model [56]. C2 uses these concerns to distinguish when a fact held from when it became queryable. A relationship learned tomorrow cannot support a decision frozen today merely because the relationship existed yesterday.

Recent designs combine timing with governance. Quipu controls graph writes while retaining validity and recording times; some policies require human approval bound to the evidence [28]. Wu's coverage model distinguishes event time from verifier receipt time, judges whether a verifier's evidence was complete at a stated cutoff, links records to decisions, and separates refused execution branches from late reports [20]. It also separates absence from one retrieval from absence in a verifier view whose closure, including retrieval coverage and retention, is justified, and it specifies that basis without designing its implementation. C2 applies these ideas to a query contract that records scope, coverage and availability outside the investigating agent.

Together, these precedents explain why C1 to C3 are a synthesis. The proposed contribution is their joint application to payment-platform investigation, with explicit channels, record classes, query responses and release rules. The hypothetical traces expose conclusions that fail when specified context is removed. E1 to E9 test parts of that context, including whether the channel states reduce bias relative to a platform's current label rule (E9) and, through planted drafts that rest only on released items, the rule that a released conclusion adds no independent corroboration (E8; manuscript Section 8.3). In S7.1, E5 and E8 measure reviewer time, and no test measures the cost of collecting the records (manuscript Section 9.1). Neither the comparison nor the traces establish that the integrated system works in deployment.

References

[1] MITRE Center for Threat-Informed Defense. 2026. Fight Fraud Framework (F3), version 1.1. GitHub, center-for-threat-informed-defense/fight-fraud-framework. Cited version: commit cb186c2 (18 September 2026). https://github.com/center-for-threat-informed-defense/fight-fraud-framework/tree/cb186c2b3372164e0b7def0d4f09531b6acc16e5. Accessed 25 October 2026.

[2] G. Dhama. 2026. Causal Label Recovery in Payment Networks. arXiv preprint 2605.29272v1. https://arxiv.org/abs/2605.29272v1.

[3] Stripe. 2026. Dispute and fraud card monitoring programs. Stripe documentation. https://docs.stripe.com/disputes/monitoring-programs. Accessed 25 October 2026.

[4] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song. 2018. Heterogeneous Graph Neural Networks for Malicious Account Detection. *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. https://doi.org/10.1145/3269206.3272010.

[5] K. Fatima, M. Schuckers, G. Cruz-Ortiz, D. Hou, S. Purnapatra, T. Andrews, A. Neupane, B. Marshall, and S. Schuckers. 2024. A Large-Scale Study of Performance and Equity of Commercial Remote Identity Verification Technologies Across Demographics. *2024 IEEE International Joint Conference on Biometrics*, pp. 1–8. https://doi.org/10.1109/IJCB62174.2024.10744432.

[6] P. Doerfler, M. Marincenko, J. Ranieri, Y. Jiang, A. Moscicki, D. McCoy, and K. Thomas. 2019. Evaluating Login Challenges as a Defense Against Account Takeover. *Proceedings of the 2019 World Wide Web Conference*, pp. 372–382. https://doi.org/10.1145/3308558.3313481.

[7] R. Sharma. 2026. Toward Auditable Fraud Detection: Combining Graph Features, Model Explanations, and Agentic Case Investigation. arXiv preprint 2607.19266v1. https://arxiv.org/abs/2607.19266v1.

[8] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr. 2025. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. arXiv preprint 2510.09023v1. https://arxiv.org/abs/2510.09023v1.

[9] P. Dash, T. Ge, A. Jain, T. Shah, and Z. Shang. 2026. From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents. arXiv preprint 2606.04329v2. https://arxiv.org/abs/2606.04329v2.

[10] Z. Xiong, Y. Lin, W. Xie, P. He, Z. Liu, J. Tang, H. Lakkaraju, and Z. Xiang. 2026. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 623–645. https://doi.org/10.18653/v1/2026.acl-long.27.

[11] H. Lakkaraju, J. Kleinberg, J. Leskovec, J. Ludwig, and S. Mullainathan. 2017. The Selective Labels Problem: Evaluating Algorithmic Predictions in the Presence of Unobservables. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 275–284. https://doi.org/10.1145/3097983.3098066.

[12] L. Bottou, J. Peters, J. Quiñonero-Candela, D. X. Charles, D. M. Chickering, E. Portugaly, D. Ray, P. Simard, and E. Snelson. 2013. Counterfactual Reasoning and Learning Systems: The

Example of Computational Advertising. *Journal of Machine Learning Research*, 14(101), pp. 3207–3260. https://jmlr.org/papers/v14/bottou13a.html.

[13] J. Singh, J. Cobbe, and C. Norval. 2019. Decision Provenance: Harnessing Data Flow for Accountable Systems. *IEEE Access*, 7, pp. 6562–6574. https://doi.org/10.1109/ACCESS.2018.2887201.

[14] A. Ehrhardt, C. Biernacki, V. Vandewalle, P. Heinrich, and S. Beben. 2021. Reject Inference Methods in Credit Scoring. *Journal of Applied Statistics*, 48(13-15), pp. 2734–2754. https://doi.org/10.1080/02664763.2021.1929090.

[15] G. Dhama. 2026. The Fundamental Limits of Fraud Detection in Card Payment Networks: An Information-Theoretic Theory of Delayed, Censored, Corrupted, and Counterfactual Feedback. arXiv preprint 2605.27557v2. https://arxiv.org/abs/2605.27557v2.

[16] A. Egg. 2025. Off-policy Evaluation for Payments at Adyen. arXiv preprint 2501.10470v1. https://arxiv.org/abs/2501.10470v1.

[17] M. Tocker. 2022. Virtual Control Group: Measuring Hidden Performance Metrics. arXiv preprint 2208.12941v1. https://arxiv.org/abs/2208.12941v1.

[18] J. Deng. 2026. When Can Fraud Operations Authorize Automation? A Decision-Support Framework for Fresh Audit Evidence and Review Workload. arXiv preprint 2608.08577v1. https://arxiv.org/abs/2608.08577v1.

[19] O. Solozobov. 2026. Decision Trace Schema for Governance Evidence in Real-Time Risk Systems. arXiv preprint 2604.09296v1. https://arxiv.org/abs/2604.09296v1.

[20] M. Wu, L. Wang, Y. Zhang, and J. Deng. 2026. Evidence Coverage for Intent-Bound Execution: Scope, Obligations, and Cutoff Reasoning. arXiv preprint 2610.01386v1. https://arxiv.org/abs/2610.01386v1.

[21] M. Manapat. 2015. Counterfactual evaluation of machine learning models. Conference talk, PyData Seattle. https://pydata.org/seattle2015/schedule/presentation/17/.

[22] R. Snodgrass and I. Ahn. 1986. Temporal Databases. *Computer*, 19(9), pp. 35–42. https://doi.org/10.1109/MC.1986.1663327.

[23] S. Razniewski and W. Nutt. 2011. Completeness of Queries over Incomplete Databases. *Proceedings of the VLDB Endowment*, 4(11), pp. 749–760. https://doi.org/10.14778/3402707.3402715.

[24] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), pp. 1792–1803. https://doi.org/10.14778/2824032.2824076.

[25] Z. Wang. 2026. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv preprint 2606.06240v1. https://arxiv.org/abs/2606.06240v1.

[26] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint 2501.13956v1. https://arxiv.org/abs/2501.13956v1.

[27] A. Dan. 2026. Anamnesis: An Authority-Preserving Runtime Contract for Persistent AI Memory. Zenodo preprint, Version 1.0, 16 August 2026. https://zenodo.org/records/22118090.

[28] S. Brown. 2026. Quipu: A Governed Bitemporal Knowledge Graph Store. arXiv preprint 2608.16813v1. https://arxiv.org/abs/2608.16813v1.

[29] J. Dai, Z. Yang, P. Shao, K. Ng, Z. Liu, C. Ju, Y. He, and B. Hu. 2026. Hearsay: Can an Auditor Trust the Record a Deployed Agent Harness Writes? arXiv preprint 2609.32495v1. https://arxiv.org/abs/2609.32495v1.

[30] J. Doyle. 1979. A Truth Maintenance System. *Artificial Intelligence*, 12(3), pp. 231–272. https://doi.org/10.1016/0004-3702(79)90008-0.

[31] B. Malheiro, N. R. Jennings, and E. Oliveira. 1994. Belief Revision in Multi-Agent Systems. *Proceedings of the 11th European Conference on Artificial Intelligence*, pp. 294–298. https://eprints.soton.ac.uk/252143.

[32] OASIS. 2021. STIX Version 2.1. OASIS Standard. https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html.

[33] Office of the Director of National Intelligence. 2015. Intelligence Community Directive 203: Analytic Standards. Directive. As amended by technical amendments through 21 January 2022. https://archive.dni.gov/files/documents/ICD/ICD-203.pdf.

[34] Y. Qi, X. Xu, and Y. Li. 2026. When Not to Write Memory: Governing False Promotion from Correlated Agent Traces. arXiv preprint 2607.02579v1. https://arxiv.org/abs/2607.02579v1.

[35] H. Akhtar and B. Freeman. 2026. Contestable Agents for Data Governance: Recording Human Stewardship of AI-Generated Metadata Across Structured and Unstructured Systems. arXiv preprint 2610.31553v1. https://arxiv.org/abs/2610.31553v1.

[36] M. Albarrak and S. Jagtap. 2026. Designing an AI-Assisted Cyber Threat Intelligence Framework for Industry 4.0: A Human-in-the-Loop Design Science Approach. *Applied Sciences*, 16(15), Article 7646. https://doi.org/10.3390/app16157646.

[37] Y. Wang, Z. Yan, J. Zhang, Z. Wu, M. Zheng, Z. Sun, Y. Zhu, and T. Cai. 2026. MAP-Graph: Provenance-Aware Shared Memory for Multi-Agent Workflows. arXiv preprint 2608.10509v1. https://arxiv.org/abs/2608.10509v1.

[38] Y. Li, Z. Xu, S. Volkov, H. Liu, T. Zhou, X. Chen, D. Wan, D. Shao, Y. Luo, and H. Sun. 2026. From Evidence to Effect: Authority Semantics and Runtime Infrastructure for Stateful Agents. arXiv preprint 2609.08472v3. Cited version: arXiv 2609.08472v3 (28 September 2026). https://arxiv.org/abs/2609.08472v3.

[39] M. Bara. 2026. Epistemic Sybil Resistance: Multiplying AI Agents Without Multiplying Evidence. arXiv preprint 2609.01873v1. https://arxiv.org/abs/2609.01873v1.

[40] G. Evron. 2026. Adversary Brains: Hypothesis-Based Logical Signatures for Threat Actor Detection with LLMs. arXiv preprint 2610.35624v1. https://arxiv.org/abs/2610.35624v1.

[41] C. Koch. 2026. From Agent Output to Authorized Transition. arXiv preprint 2609.28216v1. The v1 PDF title page reads "From Agent Output to Authorized Transition: The Agile-V Assurance Spine for State-Bound Evidence in Software, Firmware, and Hardware Engineering" instead. https://arxiv.org/abs/2609.28216v1.

[42] S. Mitra, R. Patel, S. Mittal, M. R. Rahman, and S. Rahimi. 2026. AgenticCyOps: Securing Multi-Agentic AI Integration in Enterprise Cyber Operations. arXiv preprint 2603.09134v1. https://arxiv.org/abs/2603.09134v1.

[43] A. Tausz. 2020. Similarity clustering to catch fraud rings. Stripe blog. https://stripe.com/blog/similarity-clustering. Accessed 21 October 2026.

[44] R. Drapeau and I. Khosla. 2024. The ML flywheel: how we continually improve our models to reduce card testing. Stripe blog. https://stripe.com/blog/the-ml-flywheel-how-we-continually-improve-our-models-to-reduce-card-testing. Accessed 21 October 2026.

[45] Y. Wu, M. Velazco, A. Zhao, M. R. Meléndez Luján, S. Movva, Y. K. Roy, Q. Nguyen, R. Rodriguez, Q. Wu, M. Albada, J. Kiseleva, and A. Mudgerikar. 2026. ExCyTIn-Bench: Evaluating LLM agents on Cyber Threat Investigation. arXiv preprint 2507.14201v3. Accepted at ICML 2026; final proceedings pagination not established. Cited version: arXiv 2507.14201v3 (1 May 2026). https://arxiv.org/abs/2507.14201v3.

[46] K. Mukherjee and M. Kantarcioglu. 2025. LLM-driven Provenance Forensics for Threat Investigation and Detection. arXiv preprint 2508.21323v2. The v2 PDF title page reads "LLM-driven Provenance Forensics for Threat Intelligence and Detection" instead. https://arxiv.org/abs/2508.21323v2.

[47] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang. 2026. Always-On Agents: A Survey of Persistent Memory, State, and Governance in LLM Agents. arXiv preprint 2606.30306v1. https://arxiv.org/abs/2606.30306v1.

[48] B. Xia, Q. Lu, L. Zhu, Z. Xing, D. Zhao, and H. Zhang. 2025. Evaluation-Driven Development and Operations of LLM Agents: A Process Model and Reference Architecture. arXiv preprint 2411.13768v3. https://arxiv.org/abs/2411.13768v3.

[49] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17), pp. 19632–19642. https://doi.org/10.1609/aaai.v38i17.29936.

[50] Q. Wei, T. Yang, Y. Wang, X. Li, L. Li, Z. Yin, Y. Zhan, T. Holz, Z. Lin, and X. Wang. 2025. A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv preprint 2510.02373v1. ICML 2026 acceptance reported; final proceedings version not established. Cited version: arXiv 2510.02373v1 (29 September 2025). https://arxiv.org/abs/2510.02373v1.

[51] Q. Zhan, R. Zhang, S. Guo, L. Zhao, and Z. Liu. 2026. When Memory Becomes Authority: Benchmarking Authority Collapse at the Memory Consolidation Boundary. arXiv preprint 2608.01679v2. https://arxiv.org/abs/2608.01679v2.

[52] J. Xu, Y. Xiao, W. Shao, H. Liu, and X. Li. 2026. Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory. arXiv preprint 2607.29167v1. https://arxiv.org/abs/2607.29167v1.

[53] C. Lam, J. Li, L. Zhang, and K. Zhao. 2026. Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the Stability and Safety Governed Memory (SSGM) Framework. arXiv preprint 2603.11768v2. https://arxiv.org/abs/2603.11768v2.

[54] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit. 2026. Governed Shared Memory for Multi-Agent LLM Systems. arXiv preprint 2606.24535v1. https://arxiv.org/abs/2606.24535v1.

[55] R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. Ross, P. Balaprakash, and R. Ferreira da Silva. 2025. PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. *Proceedings of the 21st IEEE International Conference on e-Science*. Cited version: arXiv 2508.02866v3 (20 August 2025). https://arxiv.org/abs/2508.02866v3.

[56] S. Kaufman, S. Rosset, and C. Perlich. 2011. Leakage in Data Mining: Formulation, Detection, and Avoidance. *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 556–563. https://doi.org/10.1145/2020408.2020496.